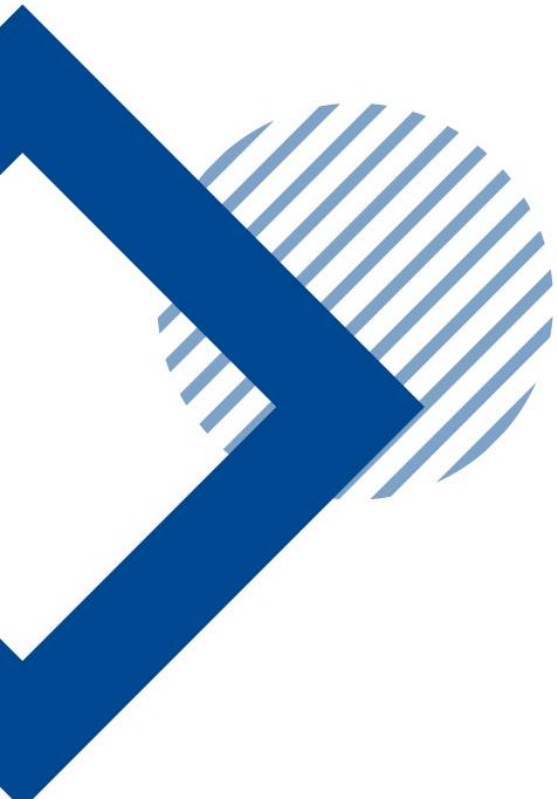
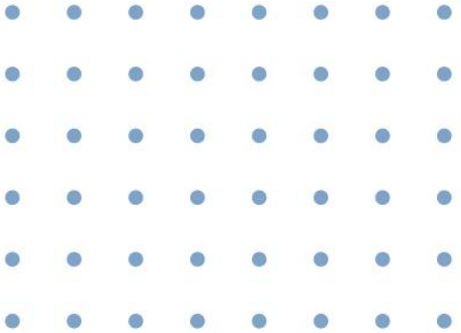
MVC

Интернет программирование







2022

MVC – 1

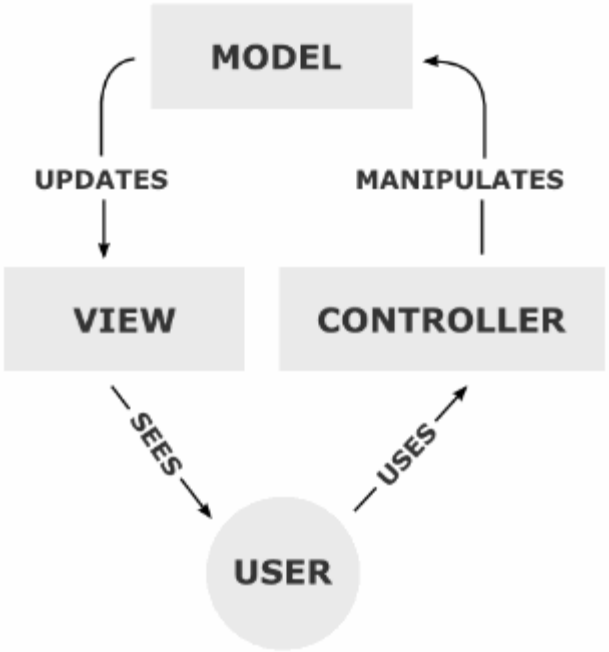
Model-View-Controller (MVC, «Модель-Представление-Контроллер», «Модель-Вид-Контроллер») — схема разделения данных приложения и управляющей логики на три отдельных компонента: модель, представление и контроллер.

Модель (Model) предоставляет данные и реагирует на команды контроллера, изменяя свое состояние.

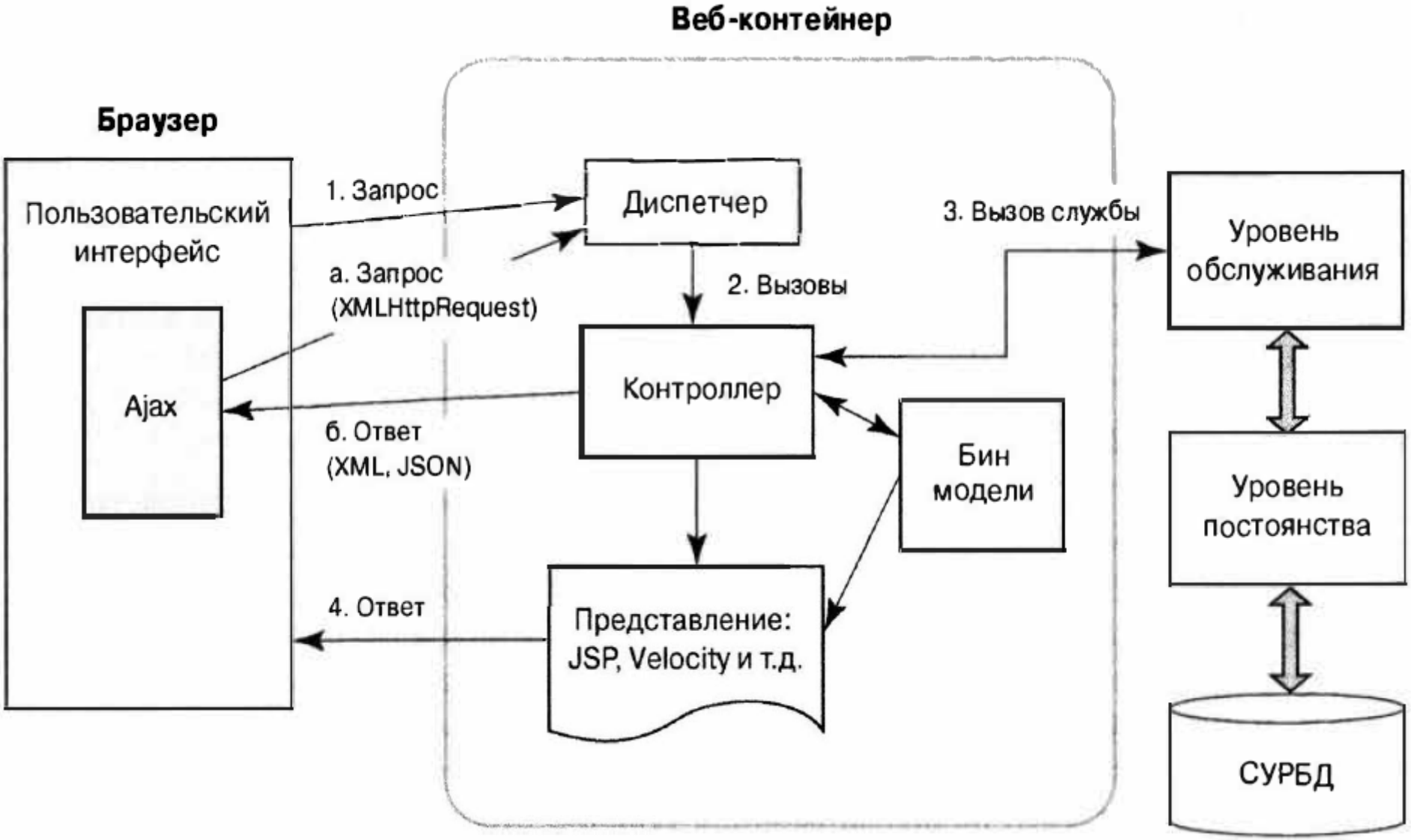
Представление (View) отвечает за отображение данных модели пользователю, реагируя на изменения модели.

Контроллер (Controller) интерпретирует действия пользователя, оповещая модель о необходимости изменений.

MVC – 2



MVC – 3


Веб-контейнер
Браузер
Пользовательский интерфейс
Ajax
1. Запрос
а. Запрос (XMLHttpRequest)
б. Ответ (XML, JSON)
4. Ответ
Диспетчер
2. Вызовы
Контроллер
Бин модели
Представление: JSP, Velocity и т.д.
3. Вызов службы
Уровень обслуживания
Уровень постоянства
СУРБД

MVC – 4

Основная цель применения MVC состоит в отделении модели от ее визуализации (представления, вида). За счет такого разделения повышается возможность повторного использования кода.

Наиболее полезно применение MVC в тех случаях, когда пользователь должен видеть одни данные одновременно с различных точек зрения.

В частности, выполняются следующие задачи:

- К одной модели можно присоединить несколько представлений, при этом не затрагивая реализацию модели.
- Не затрагивая реализацию представлений, можно изменить реакции на действия пользователя – для этого достаточно использовать другой контроллер.
- Ряд разработчиков специализируется только в одной из областей: либо разрабатывают графический интерфейс, либо разрабатывают бизнес-логику. Поэтому возможно добиться того, что программисты, занимающиеся разработкой бизнес-логики (модели), вообще не будут осведомлены о том, какое представление будет использоваться.

MVC – Модель

Модель предоставляет данные и состояние приложения внутри контекста отдельного пользователя.

Модель не зависит от представления (не знает как данные визуализировать)
и контроллера (не имеет точек взаимодействия с пользователем), просто предоставляя доступ к данным и управлению ими.

Модель строится таким образом, чтобы отвечать на запросы, изменяя свое состояние.

Модель, за счет независимости от визуального представления, может иметь несколько различных представлений.

MVC – Представление

Представление:

- **отображает данные,**
- **поддерживает взаимодействие с пользователями,**
- **осуществляет первичную проверку данных,**
- **осуществляет интернационализацию (локализация),**
- **поддерживает работу с темами оформления, и т. д.**

MVC – Контроллер

Контроллер обеспечивает «связь» между пользователем и системой.

Контролирует и направляет данные от пользователя к системе (уровню обслуживания) и наоборот.

Использует модель и представление для реализации необходимого действия.

Сервлеты Java (Servlet)

Сервлет — средство для обработки запросов.

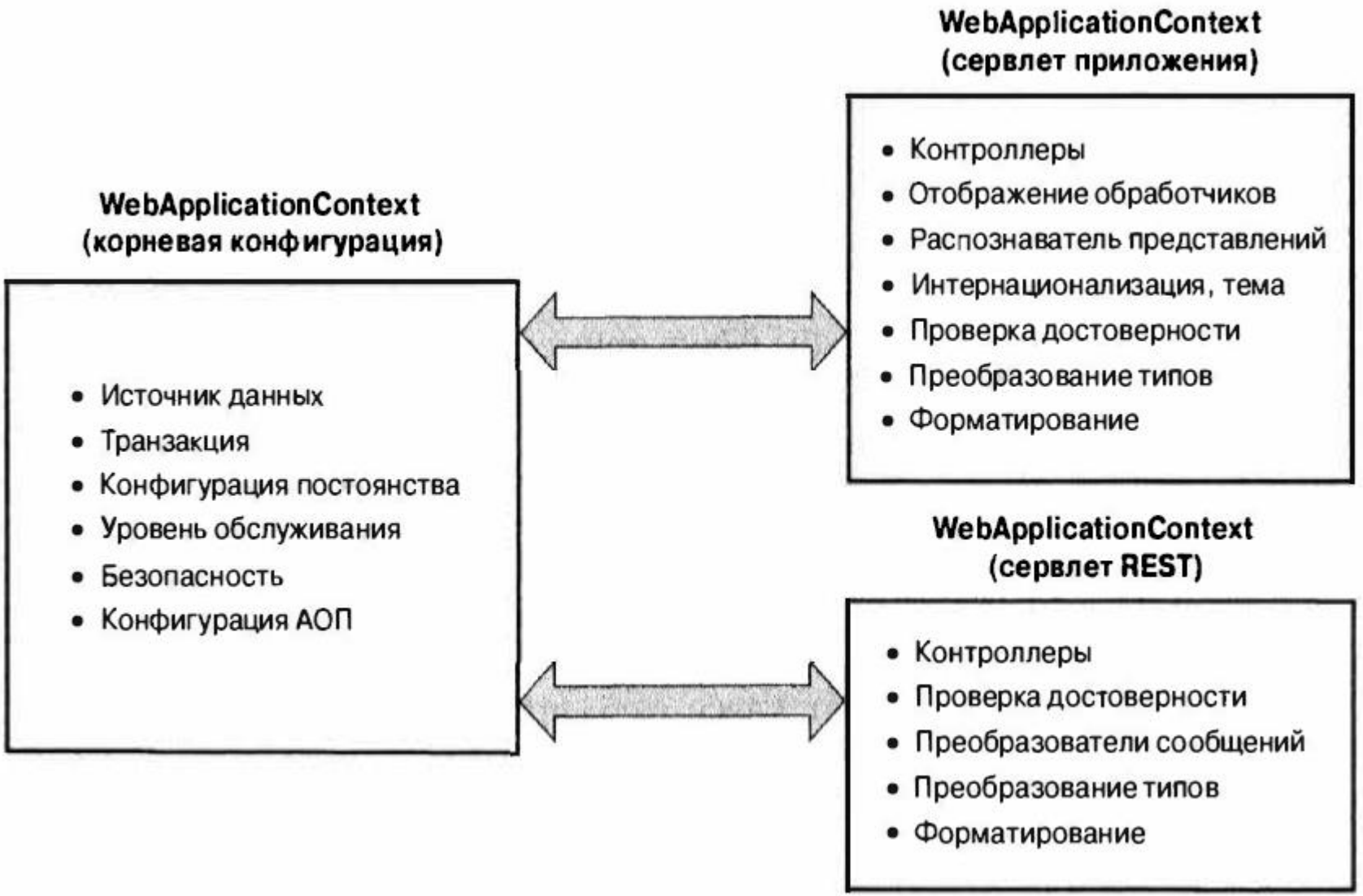
В Spring MVC основным классом сервлетов является DispatcherServlet.

В приложении может быть несколько экземпляров класса DispatcherServlet для решения отдельных задач обработки запросов.

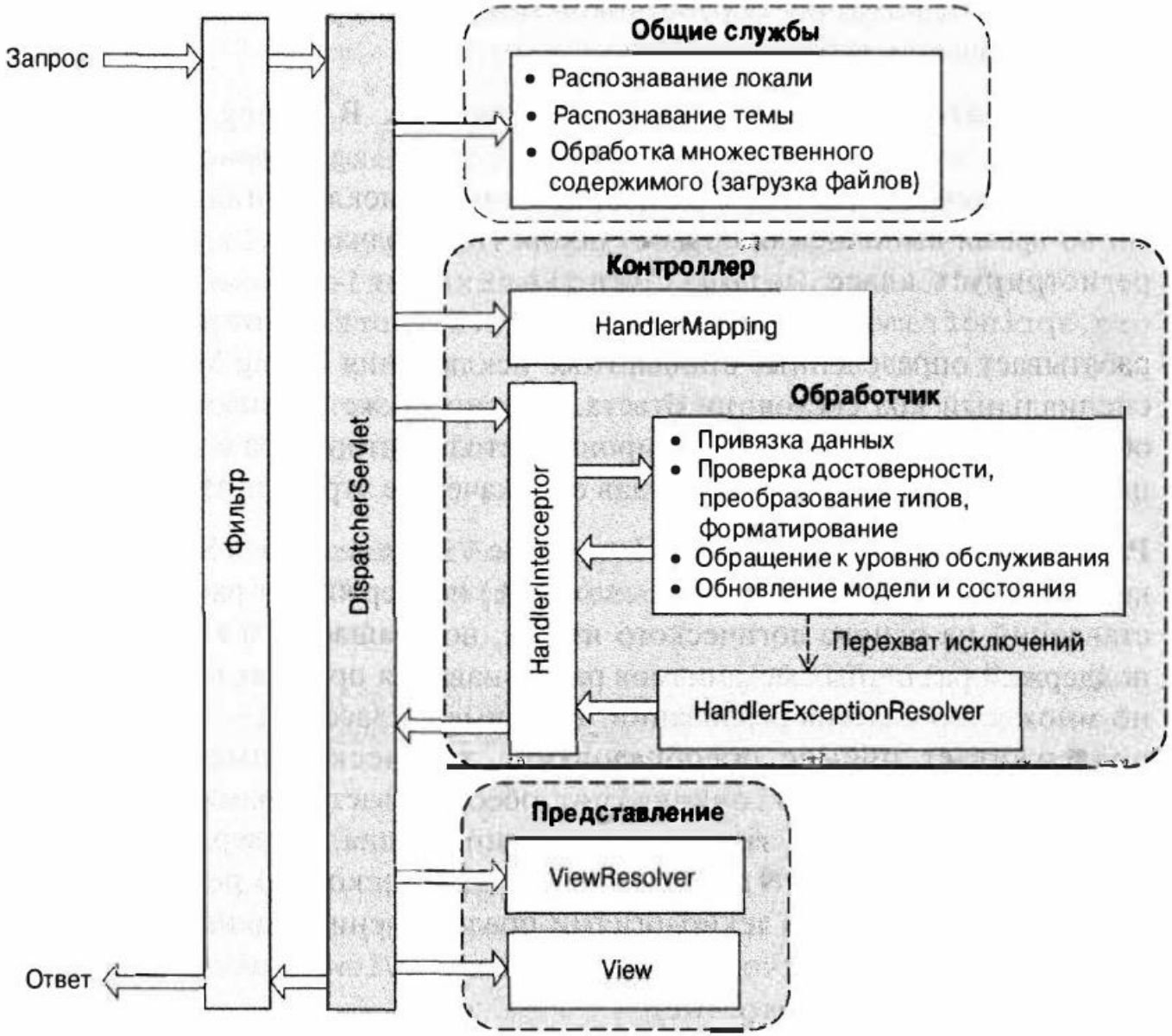
Каждый экземпляр класса DispatcherServlet имеет собственную конфигурацию WebApplicationContext.

В приложениях Spring MVC также есть корневая конфигурация WebApplicationContext.

WebApplicationContext



Жизненный цикл запроса Spring MVC



Специальные типы компонентов

DispatcherServlet использует следующие компоненты:

- HandlerMapping – связывает запрос с перехватчиком (@RequestMapping);
- HandlerAdapter – позволяет внедрить перехватчик в процесс обработки запроса;
- HandlerExceptionResolver – перехватчик ошибок;
- ViewResolver – определяет стратегию поиска представлений;
- LocaleResolver – определяет стратегию работы с ресурсами для i18n;
- ThemeResolver – определяет стратегию для работы с темами;
- MultipartResolver – определяет стратегию для работы с multi-part запросами;
- FlashMapManager – определяет стратегию передачи данных между запросами (например, при перенаправлении (redirect)).

Пример – Добавление зависимостей

build.gradle

```
...
implementation 'org.springframework.boot:spring-boot-starter-thymeleaf’
implementation 'nz.net.ultraq.thymeleaf:thymeleaf-layout-dialect:3.3.0’

runtimeOnly 'org.webjars.npm:bootstrap:5.3.3'
runtimeOnly 'org.webjars.npm:bootstrap-icons:1.11.3’
...
```

Пример – Контроллер

```
@Controller
@RequestMapping(TypeController.URL)
public class TypeController {

}
```

Пример – Контроллер - Read

```
@GetMapping
 public String getAll(Model model) {
    model.addAttribute(
          "items",
          typeService.getAll().stream()
                .map(this::toDto)
                .toList());
    return TYPE_VIEW;
}
```

Пример – Контроллер - Create

```
@GetMapping("/edit/")
public String create(Model model) {
    model.addAttribute(TYPE_ATTRIBUTE, new TypeDto());
    return TYPE_EDIT_VIEW;
}

@PostMapping("/edit/")
public String create(
        @ModelAttribute(name = TYPE_ATTRIBUTE) @Valid TypeDto type,
        BindingResult bindingResult,
        Model model) {
    if (bindingResult.hasErrors()) {
        return TYPE_EDIT_VIEW;
    }
    typeService.create(toEntity(type));
    return Constants.REDIRECT_VIEW + URL;
}
```

Пример – Контроллер - Update

```
@GetMapping("/edit/{id}")
public String update(
        @PathVariable(name = "id") Long id,
        Model model) {
    if (id <= 0) {
        throw new IllegalArgumentException();
    }
    model.addAttribute(TYPE_ATTRIBUTE, toDto(typeService.get(id)));
    return TYPE_EDIT_VIEW;
}

@PostMapping("/edit/{id}")
public String update(
        @PathVariable(name = "id") Long id,
        @ModelAttribute(name = TYPE_ATTRIBUTE) @Valid TypeDto type,
        BindingResult bindingResult,
        Model model) {
    if (bindingResult.hasErrors()) {
        return TYPE_EDIT_VIEW;
    }
    if (id <= 0) {
        throw new IllegalArgumentException();
    }
    typeService.update(id, toEntity(type));
    return Constants.REDIRECT_VIEW + URL;
}
```

Пример – Контроллер - Delete

```
@PostMapping("/delete/{id}")
public String delete(
        @PathVariable(name = "id") Long id) {
    typeService.delete(id);
    return Constants.REDIRECT_VIEW + URL;
}
```

Пример – Контроллер - Cookie

```
@PostMapping(LOGIN_URL)
// Порядок аргументов важен!
public String login(
        HttpServletResponse response,
        @ModelAttribute(name = USER_ATTRIBUTE) @Valid UserLoginDto user,
        BindingResult bindingResult,
        Model model) {
    if (bindingResult.hasErrors()) {
        return LOGIN_VIEW;
    }
    final UserEntity existsUser = userService.getByLogin(user.getLogin());
    final Cookie cookie = new Cookie(Constants.USER_COOKIE, existsUser.getId().toString());
    response.addCookie(cookie);
    return Constants.REDIRECT_VIEW + "/";
}

@GetMapping
public String getProfile(
        @RequestParam(name = "page", defaultValue = "0") int page,
        @CookieValue(value = Constants.USER_COOKIE, defaultValue = "0") long userId,
        Model model) {
   ... orderService.getTotal(userId) ...
 }
```

Пример – Контроллер – Сессии 1

```
@Controller
@RequestMapping(UserCartController.URL)
@SessionAttributes("types")
public class UserCartController {

    @GetMapping
    public String getCart(Model model) {
        ...
        model.addAttribute("types", types);
        ...
     }

   @PostMapping
    public String addOrderToCart(
            @ModelAttribute(name = ORDER_ATTRIBUTE) @Valid CartDto order,
            BindingResult bindingResult,
            SessionStatus status,
            Model model) {
        if (bindingResult.hasErrors()) {
            return ORDER_VIEW;
        }
        status.setComplete();
        ...
    }
```

Пример – Контроллер – Сессии 2

```
public class Cart extends HashMap<Integer, CartDto> {
}

@Bean
@Scope(value = WebApplicationContext.SCOPE_SESSION, proxyMode = ScopedProxyMode.TARGET_CLASS)
Cart todos() {
    return new Cart();
}

@Controller
public class UserCartController {
    private final Cart cart;

    @PostMapping("/clear")
    public String clearCart() {
        cart.clear();
        return Constants.REDIRECT_VIEW + URL;
    }
}
```

Пример – Глобальный контроллер

```
@ControllerAdvice
public class GlobalController {
    @ModelAttribute("servletPath")
    String getRequestServletPath(HttpServletRequest request) {
        return request.getServletPath();
    }

    @ModelAttribute("totalCart")
    double getTotalCart(HttpSession session) {
        return cart.getSum();
    }

    @ModelAttribute("userName")
    String getUserName(@CookieValue(value = Constants.USER_COOKIE, defaultValue = "0") long userId) {
        if (userId <= 0) {
            return "";
        }
        return userService.get(userId).getLogin();
    }
}
```

Пример – Шаблон

default.html

```
<head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title layout:title-pattern="$LAYOUT_TITLE - $CONTENT_TITLE">My shop</title>
    <script type="text/javascript" src="/webjars/bootstrap/5.3.3/dist/js/bootstrap.bundle.min.js"></script>
    <link rel="stylesheet" href="/webjars/bootstrap/5.3.3/dist/css/bootstrap.min.css" />
    <link rel="stylesheet" href="/webjars/bootstrap-icons/1.11.3/font/bootstrap-icons.min.css" />
    <link rel="stylesheet" href="/css/style.css" />
</head>
```

Пример – Шаблон (основной)

```
<th:block th:switch="${items.size()}">
      <h2 th:case="0">Данные отсутствуют</h2>

      <th:block th:case="*">
      </th:block>
 </th:block>
            <div>
              <a href="/admin/type/edit/" class="btn btn-primary">Добавить тип заказа</a>
            </div>
           <tbody>
              <tr th:each="type : ${items}"> <!-- public String getAll(Model model) -->
                 <th scope="row" th:text="${type.id}"></th>
                 <td th:text="${type.name}"></td>
                 <td>
                   <form th:action="@{/admin/type/edit/{id}(id=${type.id})}" method="get">
                      <button type="submit" class="btn btn-link button-link">Редактировать</button>
                   </form>
                 </td>
                 <td>
                   <form th:action="@{/admin/type/delete/{id}(id=${type.id})}" method="post">
                      <button type="submit" class="btn btn-link button-link"
                         onclick="return confirm('Вы уверены?')">Удалить</button>
                   </form> <!-- public String delete(@PathVariable(name = "id") Long id) -->
                 </td>
              </tr>
           </tbody>
```

Пример – Шаблон (create/update)

```
<form action="#" th:action="@{/admin/type/edit/{id}(id=${type.id})}" th:object="${type}" method="post">
    <div class="mb-3">
        <label for="id" class="form-label">ID</label>
        <input type="text" th:value="*{id}" id="id" class="form-control" readonly disabled>
    </div>
    <div class="mb-3">
        <label for="name" class="form-label">Тип заказа</label>
        <input type="text" th:field="*{name}" id="name" class="form-control">
        <div th:if="${#fields.hasErrors('name')}" th:errors="*{name}" class="invalid-feedback"></div>
    </div>
    <div class="mb-3 d-flex flex-row">
        <button class="btn btn-primary me-2 button-fixed-width" type="submit">Сохранить</button>
        <a class="btn btn-secondary button-fixed-width" href="/admin/type">Отмена</a>
    </div>
</form>
```

```
public String create(
    @ModelAttribute(name = TYPE_ATTRIBUTE) @Valid TypeDto type,
    BindingResult bindingResult,
    Model model)

public String update(
    @PathVariable(name = "id") Long id,
    @ModelAttribute(name = TYPE_ATTRIBUTE) @Valid TypeDto type,
    BindingResult bindingResult,
    Model model) {
```

Пример – Шаблон (layout)

default.html

```
<html>
<head></head>

<body>
    <main class="container-fluid p-2" layout:fragment="content">
    </main>
</body>
</html>
```

someView.html

```
<html layout:decorate="~{default}">
<head></head>

<body>
   <main layout:fragment="content">
      ....
   </main>
</body>
</html>
```

Пример – Шаблон (компоненты)

pagination.html

```
<html>

<body>
   <th:block th:fragment="pagination (url, totalPages, currentPage)">
   </th:block>

</body>

</html>
```

someView.html

```
<html layout:decorate="~{default}">
<head></head>

<body>
   <main layout:fragment="content">
      <th:block th:replace="~{ pagination :: pagination (
               url=${'admin/user'},
               totalPages=${totalPages},
               currentPage=${currentPage}) }" />
   </main>
</body>
</html>
```

Пример – Шаблон (синтаксис 1)

```
<th:block th:if="${#strings.isEmpty(message)}">
    <li class="list-group-item">
        Неизвестная ошибка
    </li>
</th:block>


 <th:block th:if="${not #strings.isEmpty(url)}">
   <li class="list-group-item">
       [[${method}]] ([[${file}]]:[[${line}]])
    </li>
</th:block>


<footer class="my-footer mt-auto d-flex flex-shrink-0 justify-content-center align-items-center">
     Автор, [[${#dates.year(#dates.createNow())}]]
</footer>


<ul class="navbar-nav me-auto link" th:with="activeLink=${#objects.nullSafe(servletPath, '')}">
    <a class="nav-link" href="/admin/user"
            th:classappend="${activeLink.startsWith('/admin/user') ? 'active' : ''}">
            Пользователи
     </a>
```

Пример – Шаблон (синтаксис 2)

```
<div class="mb-2 col-12 col-md-8 col-lg-6 d-flex justify-content-center"
   th:if="${not #lists.isEmpty(cart)}">


 <form action="#"
       th:action=
       "@{/cart/decrease?type={type}&price={price}(type=${cartItem.type},price=${cartItem.price})}"
       method="post">
       <button class="btn btn-danger">
           <i class="bi bi-dash-lg"></i>
        </button>
  </form>


 <select th:field="*{type}" id="type" class="form-select">
        <option selected value="">Укажите тип товара</option>
        <option th:each="type : ${types}" th:value="${type.id}">[[${type.name}]]</option>
  </select>
```

Спасибо за внимание!

Литература:
1. Spring Boot CRUD Application with Thymeleaf https://www.baeldung.com/spring-boot-crud-thymeleaf
2. Thymeleaf Layout Dialect https://ultraq.github.io/thymeleaf-layout-dialect/
3. Tutorial: Using Thymeleaf https://www.thymeleaf.org/doc/tutorials/3.1/usingthymeleaf.html#introducing-thymeleaf
4. Working with Cookies in Spring MVC using @CookieValue Annotation https://www.geeksforgeeks.org/working-with-cookies-in-spring-mvc-using-cookievalue-annotation/
5. Session Attributes in Spring MVC https://www.baeldung.com/spring-mvc-session-attributes



2022